Approximate Nearest Neighbor Search with Vamana and DiskANN

Deadline: 16th October, 2026

In this assignment, you have to perform the following tasks:

- Prepare fixed-dimensional vector datasets for approximate nearest-neighbor search.
- Run exact nearest-neighbor search and HNSW using FAISS.
- Implement the Vamana graph index underlying DiskANN.
- Study how Vamana's parameters affect the recall-latency trade-off.
- Compare your implementation against FAISS HNSW.
- Extra credit: implement disk-resident search in the style of DiskANN and benchmark HDD versus SSD when both are available.

The compulsory portion of the assignment concerns in-memory Vamana. Disk-resident search is reserved for extra credit.

Datasets

Use the following three datasets for the main assignment.

SIFT1M. Use the standard SIFT1M approximate nearest-neighbor benchmark: approximately one million database vectors, 128 dimensions, supplied query vectors, and supplied exact-nearest-neighbor ground truth. Use Euclidean (L2) distance.

Quora and Natural Questions (NQ). Use the test-query split of the Quora and NQ collections from the BEIR benchmark. Obtain the precomputed single-vector query and corpus embeddings from CohereLabs/beir-embed-english-v3. The available corpus sizes are approximately 523K vectors for Quora and 2.68M vectors for NQ. Each original embedding is 1024-dimensional.

Dimensionality reduction for the IR datasets

For Quora and NQ, reduce the supplied 1024-dimensional embeddings to 128 dimensions using PCA. Perform the following steps separately for each dataset:

- Fit PCA using only corpus embeddings. Do not include query embeddings while fitting PCA.
- You may fit PCA on a uniform random sample of the corpus rather than the complete corpus. Report the sample size and random seed.
- Apply the same fitted PCA transformation to both corpus and query embeddings.
- Cast the projected vectors to float32.
- L2-normalize the projected corpus and query vectors.
- Use these 128-dimensional vectors for all subsequent exact and approximate nearest-neighbor experiments.

Do not fit separate PCA transformations for queries and corpus documents. Standard PCA implementations, including randomized or incremental variants, are allowed.

Exact-neighbor ground truth and recall

The BEIR relevance judgments are not the ground truth used in this assignment. After PCA and normalization, use exact FAISS search over the 128-dimensional corpus vectors to obtain the true nearest neighbors for each query. Save these exact results and reuse them throughout the assignment.

Following the terminology used in the DiskANN paper, report 1-recall@1 and 10-recall@10. For k-recall@k, if $G_k(q)$ is the exact top-k set and $A_k(q)$ is the approximate top-k set, compute $|A_k(q) \cap G_k(q)| / k$ and average over queries.

For SIFT1M, use the supplied exact ground truth. For Quora and NQ, FAISS exact search over the projected 128-dimensional vectors defines the exact ground truth. Do not evaluate 128-dimensional ANN results against neighbors computed in the original 1024-dimensional space.

> ***Report deliverable:*** *one dataset-summary table containing corpus size, number of queries, original dimension, search dimension, and raw float32 memory required by the 128-dimensional corpus vectors. For Quora and NQ, also state the PCA implementation, fitting-sample size, random seed, and variance explained by the retained 128 components.*

Part 1: Exact Search and HNSW with FAISS

You are not required to implement either baseline yourself. Use FAISS for exact nearest-neighbor search and HNSW.

1a. Exact kNN. Use an exact FAISS index such as IndexFlatL2. For SIFT1M, verify that the result agrees with the supplied ground truth. For Quora and NQ, compute the exact top-k results once, save them, and use them as the reference answer sets for all subsequent recall calculations.

1b. HNSW. Use FAISS IndexHNSWFlat. Choose sensible construction parameters such as M and efConstruction, report them, and obtain at least five query-time operating points by varying efSearch. Exhaustive hyperparameter tuning is not required.

For timing, use the same CPU-thread setting and query mode for all methods being compared. Where a dataset has more than 1000 test queries, you may use a fixed random subset of at least 1000 queries for timing only; recall should be computed over the full evaluation query set unless computation becomes prohibitive. Clearly state any deviation.

> ***Report deliverable:*** *for every dataset, report 1-recall@1, 10-recall@10, mean latency/query, QPS, build time, and index size for exact FAISS and at least five HNSW efSearch settings. Provide one HNSW 10-recall@10 vs. mean-query-latency plot per dataset. State CPU model, CPU thread count, and whether queries were issued individually or in batches.*

Part 2: Implement Vamana

You will now implement Vamana, the graph-based ANN index underlying DiskANN. Read the DiskANN paper before starting this part:
DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node

You must implement the following yourself:

- GreedySearch
- RobustPrune
- Vamana graph construction

You may use NumPy, PyTorch, Numba, Cython, C/C++, BLAS/OpenMP, multiprocessing, priority queues, and similar tools for efficient numerical operations and data structures. You may not call an existing Vamana or DiskANN implementation for the compulsory portion of the assignment.

2a. GreedySearch

Given a directed graph G, entry vertex s, query vector q, requested result count k, and search-list size L, maintain a bounded candidate set and repeatedly expand the closest unvisited candidate:

```
GreedySearch(G, s, q, k, L):
    C <- {s}
    V <- empty set

    while C minus V is not empty:
        p <- point in C minus V closest to q
        V <- V union {p}
        C <- C union OutNeighbors(p)

        if |C| > L:
            retain in C only the L points closest to q

    result <- the k points in C closest to q
    return result, V
```

The complete visited set V, not merely the final k results, is needed during Vamana construction. Your implementation should count both graph vertices expanded and previously unseen query-to-database distances evaluated.

2b. RobustPrune

For point p, candidate set V, maximum out-degree R, and pruning parameter $\alpha \geq 1$, use the following procedure:

```
RobustPrune(G, p, V, alpha, R):
    V <- (V union OutNeighbors(p)) minus {p}
    OutNeighbors(p) <- empty set

    while V is not empty:
        p_star <- point in V closest to p
        add p_star to OutNeighbors(p)
        remove p_star from V

        if |OutNeighbors(p)| == R:
            break

        for each p_prime in V:
            if alpha * d(p_star, p_prime) <= d(p, p_prime):
                remove p_prime from V
```

Increasing α makes the pruning condition harder to satisfy, allowing additional long-range graph connections to survive.

2c. Vamana graph construction

Initialize a random R-regular directed graph. Use the database point nearest the dataset mean as a practical approximation to the medoid/entry point. Perform two passes over the data: first with $\alpha = 1$, then with the chosen $\alpha \geq 1$.

```
BuildVamana(P, R, L_build, alpha):
    initialize G as a random R-regular directed graph

    centroid <- mean vector of P
    s <- point in P closest to centroid

    for alpha_pass in [1, alpha]:
        order <- random permutation of P

        for p in order:
            _, visited <- GreedySearch(G, s, vector[p], 1, L_build)
            RobustPrune(G, p, visited, alpha_pass, R)

            for each j in OutNeighbors(p):
                C <- OutNeighbors(j) union {p}

                if |C| > R:
                    RobustPrune(G, j, C, alpha_pass, R)
                else:
                    OutNeighbors(j) <- C

    return G, s
```

At query time, use GreedySearch from s with a separate query-time parameter L_search. Do not confuse L_build with L_search.

Use R = 64, L_build = 100, and α = 1.2 as an initial reference configuration. You may change these values if necessary, but report and justify any change.

Performance and implementation

The full datasets contain hundreds of thousands to millions of vectors. A correct but naïve Python implementation may be prohibitively slow. Performance optimization is expected and encouraged.

- Keep vectors in contiguous float32 arrays.
- Batch/vectorize distance calculations over neighbor sets; avoid Python-level loops over vector dimensions or one NumPy call per individual neighbor distance.
- Use compact integer arrays for graph adjacency where practical.
- Avoid unnecessary copies and repeated allocation of large temporary arrays.
- Reuse search buffers and visitation structures where practical.
- Use efficient priority-queue/candidate-set implementations.
- Use Numba, Cython, C/C++, OpenMP, multiprocessing, or similar acceleration where useful.
- Parallelize independent work where appropriate.

The implementations of GreedySearch, RobustPrune, and Vamana itself must nevertheless be your own.

Development procedure

Before launching full builds, test on subsets and measure scaling. At minimum, obtain construction-time measurements around 100K, 250K, and 500K points where possible. Use these measurements to estimate whether your implementation is ready for the full dataset.

For parameter debugging and ablations in Part 3, create a SIFT100K development set from SIFT1M and compute its exact top-k neighbors with FAISS.

> ***Report deliverable:*** *for SIFT1M, Quora, and NQ, report R, L_build, α, total build time, average out-degree, maximum out-degree, serialized graph/index size, and peak build memory. State the entry-point procedure, random seed, graph representation, programming language/libraries used for the performance-critical code, and the major optimizations you used. Verify that no vertex has out-degree greater than R. For one SIFT100K query, additionally report exact top-10 IDs, Vamana top-10 IDs, 10-recall@10, vertices expanded, and distance evaluations.*

Part 3: Vamana Parameter Study and Comparison with HNSW

3a. Parameter study on SIFT100K

Use SIFT100K for construction-time parameter experiments so that you do not repeatedly rebuild million-point graphs.

Start from R = 64, L_build = 100, α = 1.2. Construct at least the reference graph, one graph with a different R, and one graph with a different α. You do not need to evaluate the full Cartesian product.

For each constructed graph, vary L_search over at least five values (for example 10, 20, 50, 100, 200) to obtain multiple recall-latency operating points.

> ***Report deliverable:*** *for every Vamana operating point on SIFT100K, report R, α, L_build, L_search, 1-recall@1, 10-recall@10, mean latency, QPS, distance evaluations/query, and vertices expanded/query. Provide (i) 10-recall@10 vs. L_search and (ii) 10-recall@10 vs. mean latency. Discuss the effects of L_search, R, and α, including why a graph with more outgoing edges may sometimes require fewer graph expansions to reach a target recall.*

3b. Full-dataset comparison

For SIFT1M, Quora, and NQ, build one main Vamana index using your chosen construction parameters. Do not repeatedly rebuild the full datasets merely to generate parameter sweeps. Vary only L_search on each completed full graph.

Compare FAISS exact search, FAISS HNSW, and your Vamana implementation. Exact search serves both as the ground-truth generator/reference point and as a timing reference; HNSW is the approximate baseline.

> ***Report deliverable:*** *for each full dataset, provide one 10-recall@10 vs. mean-query-latency plot with Vamana and FAISS HNSW on the same axes and exact FAISS shown as a reference point. Give a compact table containing 1-recall@1, 10-recall@10, latency, QPS, build time, and index size for exact FAISS and representative comparable-recall HNSW/Vamana settings. For Vamana also report average distance evaluations/query and vertices expanded/query. Discuss recall-latency, build-time, and index-size trade-offs.*

Evaluation Procedure

Use the same 128-dimensional vectors and the same distance metric for every method on a dataset. The principal quality metrics are:

- **1-recall@1**
- **10-recall@10**

These are nearest-neighbor fidelity metrics: they measure agreement with exact vector search, and we are not measuring relevance according to the original BEIR qrels.

For normal in-memory timing:

- Perform a short warm-up before timing.
- Exclude one-time index loading from query latency.
- Report CPU model and number of threads.
- Use the same threading and query-batching policy when comparing HNSW and Vamana.
- Repeat timing runs when necessary to obtain stable measurements.
- Save exact-neighbor results after computing them once.

Extra Credit (4 points total)

1. Disk-Resident DiskANN Search (2 points). The compulsory assignment keeps the Vamana graph and vectors in memory. For this task, implement a genuinely disk-resident search system inspired by DiskANN.

Use the full MS MARCO passage embedding collection from the same Cohere BEIR embedding source. You may again reduce the 1024-dimensional embeddings to 128 dimensions using PCA. Impose a maximum 8 GiB RAM budget on the ANN index/search structures. The full graph and full-precision vector collection must not simply be loaded into RAM before querying.

Your design should include:

- a compact in-memory representation used to guide search;
- graph neighborhoods and full-precision vectors stored on disk;
- direct lookup of disk records by vertex ID;
- on-demand fetching of graph neighborhoods during search;
- a small beam of promising vertices so that several disk reads can be processed together;
- a sufficiently accurate/full-precision final ranking.

You may use an existing Product Quantization implementation such as FAISS PQ; you are not required to implement the quantizer yourself. For this extra-credit task you may also use your own Part 2 graph builder or an existing/instructor-approved builder. The work being evaluated is the disk-resident search organization and search procedure.

Cold- and warm-cache measurement

By cold and warm cache, we mean if the pages of the index have been brought into page cache before execution or not. For cold-cache timing, do not merely assume the index is uncached. Use Linux page-cache tools to verify and evict the relevant index files on a per-file basis.

1. Close/unmap the index files.
2. Use fincore to inspect their page-cache residency.
3. Use fadvise with DONTNEED (or an equivalent per-file mechanism) to advise the OS that the cached index pages are no longer needed.
4. Use fincore again and record the resulting residency.
5. Start the search process and run the timed workload.

File-cache eviction is advisory, so report the residency observed by fincore immediately before the cold run. You need not use system-wide /proc/sys/vm/drop_caches as the normal experimental procedure. Also perform a separate warm-cache experiment after sufficiently exercising the index.

> ***Report deliverable:*** *describe the disk-record layout, graph representation, what remains permanently resident in RAM, compressed-vector representation if used, beam width, imposed RAM limit, measured peak RAM, storage device, and cold-cache procedure. For cold and warm runs report 1-recall@1, 10-recall@10, mean/p50/p95 latency, QPS, peak RAM, reads/query, bytes read/query, total on-disk index size, fincore residency before the cold run, graph expansions/query, and disk-search rounds/query. A system that loads the complete index into RAM before search receives no credit.*

2. HDD vs. SSD DiskANN (2 points). This is a follow-up to Extra Credit 1. If you have access to both a physical HDD and a physical SSD, place an identical copy of the same disk index on each device and run the same search workload.

Keep the graph, vector representation, query set, memory limit, beam width, and all search parameters fixed. The only intended experimental variable is the storage device. Run both cold-cache and warm-cache measurements using the same fincore/fadvise protocol.

Do not treat two partitions of the same physical drive as an HDD/SSD comparison. Students are not expected to purchase hardware solely for this task.

> ***Report deliverable:*** *report the exact HDD and SSD models/interfaces and, for HDD-cold, HDD-warm, SSD-cold, and SSD-warm, report 10-recall@10, mean/p50/p95 latency, QPS, reads/query, bytes/query, and disk-search rounds/query. Provide one 10-recall@10 vs. mean-latency plot for HDD and SSD. Discuss why graph search produces a difficult random-access pattern, why cold HDD and SSD behavior differ, how much of the difference remains when warm, and whether the bottleneck appears to be storage latency, cache/memory behavior, or distance computation.*

General Hints

- Save exact-neighbor results after computing them once.
- Persist completed Vamana/HNSW indices instead of rebuilding them for every experiment.
- Test Vamana thoroughly on SIFT100K before launching million-point builds.
- Measure construction time on successively larger subsets before starting NQ.
- Use vectorized/batched distance calculations aggressively.
- Use float32 vectors and compact integer IDs for graph adjacency.
- Monitor actual peak memory, not just the theoretical vector-matrix size.
- Performance optimization is part of making a scalable ANN implementation; compiled/vectorized tools are encouraged.

Submission Protocol

Moodle OR Google Forms (Drive upload) will be used for submission.

- All findings go into a single report file, structured to mirror the assignment parts.
- Zip all implementation and experiment scripts together with the report into a single file named [Roll1_Roll2].(zip/tar.gz).
- Do not include downloaded datasets or embedding collections inside the submission archive.
- Only one team member submits.

Summary of Report Deliverables

- Dataset preparation — corpus/query sizes, PCA procedure, sample size/seed, explained variance, resulting 128-d memory footprint.
- Part 1 — exact FAISS + HNSW metrics and HNSW recall-latency plot for all datasets.
- Part 2 — Vamana implementation details, performance optimizations, full-dataset build statistics, and SIFT100K sanity check.
- Part 3a — SIFT100K R / α / L_search study with required tables and plots.
- Part 3b — Vamana vs. HNSW on SIFT1M, Quora, and NQ at comparable recall.
- Extra Credit 1 — disk-resident search, RAM/disk organization, and cold/warm measurements with page-cache residency evidence.
- Extra Credit 2 — identical-index HDD vs. SSD comparison with cold/warm latency analysis.

LLM Usage Policy

Any use of LLM assistants such as Claude, ChatGPT, Gemini, etc. needs to be documented in full length in your reports. You will also have to provide shareable links to all chats which you used to implement this programming assignment. Using these assistants and not providing this information goes against the course's honor code.